SAECOIN — CONTRIBUTION GUIDE

Document: Contribution_Guide.md
Project: SAECOIN
Version: 1.0
Status: Draft
Repository: Official SAECOIN GitHub Repository (To Be Created)
License: To Be Decided

---

1. PURPOSE

This document defines the contribution rules for developers, testers, researchers, and community members who want to contribute to SAECOIN.

The objectives are to:

- Maintain consistent code quality.
- Protect blockchain consensus integrity.
- Encourage transparent development.
- Prevent unauthorized changes.
- Improve testing and documentation.
- Support community collaboration.
- Maintain project security.

All contributors must follow these guidelines.

---

2. CONTRIBUTION AREAS

Contributions may include:

2.1 Blockchain Core

- Rust blockchain implementation.
- Block validation.
- Transaction validation.
- UTXO management.
- Consensus implementation.
- Difficulty adjustment.
- P2P networking.

2.2 Mining

- CPU mining.
- GPU mining.

- Mining performance optimization.
- Benchmarking.
- Mining documentation.

2.3 Wallet Development

- Android wallet.
- Windows desktop wallet.
- Wallet recovery.
- Transaction signing.
- User interface improvements.
- Wallet security testing.

2.4 Explorer and API

- Blockchain explorer.
- Public APIs.
- Node monitoring.
- Network statistics.
- Developer tools.

2.5 Testing and Security

- Unit tests.
- Integration tests.
- Fuzz testing.
- Performance testing.
- Security research.
- Protocol review.

2.6 Documentation

- Technical documentation.
- User guides.
- Developer guides.
- Translation.
- Troubleshooting instructions.

---

3. DEVELOPMENT PRINCIPLES

All contributions should follow these principles:

1. Security before convenience.
2. Consensus correctness before performance.
3. Readable code before unnecessary complexity.
4. Automated testing wherever practical.

5. Transparent development.
6. Minimal unnecessary dependencies.
7. Backward compatibility where required.
8. Clear documentation for important changes.

Contributors must not introduce undocumented protocol changes.

---

4. GETTING STARTED

Before contributing, developers should:

1. Read the project README.
2. Read the Documentation Index.
3. Review the Technical Requirements Document.
4. Read the Coding Standards.
5. Understand the repository structure.
6. Set up the required development environment.
7. Build the project locally.
8. Run the existing tests.

The final repository setup instructions must be maintained in the project README.

---

5. GITHUB WORKFLOW

The recommended contribution workflow is:

1. Fork the official repository, if external contributions are enabled.
2. Clone the repository.
3. Create a dedicated branch.
4. Implement the proposed change.
5. Add or update tests.
6. Run formatting and validation checks.
7. Update documentation where necessary.
8. Commit the changes.
9. Open a Pull Request.
10. Respond to review feedback.

External contributions must not be merged directly into protected main branches.

---

6. BRANCH NAMING

Recommended branch conventions:

Branch Prefix| Purpose
feature/| New features
fix/| Bug fixes
security/| Security-related fixes
consensus/| Consensus-related work
test/| Testing improvements
docs/| Documentation
refactor/| Code restructuring
performance/| Performance optimization

Examples:

"feature/wallet-backup"

"fix/transaction-validation"

"security/p2p-message-validation"

"docs/node-setup"

---

7. COMMIT MESSAGE RULES

Commit messages should be clear and descriptive.

Recommended format:

"type: short description"

Examples:

"feat: add wallet backup support"

"fix: reject invalid transaction inputs"

"test: add UTXO validation tests"

"docs: update node setup instructions"

"refactor: simplify block validation"

Avoid vague messages such as:

- update
- changes
- fix stuff

- final version

---

8. PULL REQUEST REQUIREMENTS

Every Pull Request should include:

- A clear title.
- Description of the change.
- Reason for the change.
- Affected components.
- Relevant test results.
- Documentation updates.
- Known limitations.

For consensus-related changes, additionally include:

- Exact rule being changed.
- Compatibility implications.
- Test vectors.
- Historical block validation results.
- Reorganization implications.
- Security analysis.

A Pull Request must not be merged solely because the code compiles.

---

9. CONSENSUS CHANGE POLICY

Consensus changes are especially sensitive.

Examples:

- Block reward modifications.
- Supply limit changes.
- Difficulty algorithm changes.
- Transaction validation changes.
- Signature verification changes.
- Block format changes.
- Chain-selection changes.

Rules:

1. Consensus changes require a written technical proposal.
2. Existing consensus behavior must be documented.
3. Changes must include comprehensive tests.

4. Independent review is required before production adoption.
5. Compatibility implications must be analyzed.
6. Mainnet activation rules must be explicitly specified.
7. No individual contributor may silently change consensus rules.

A code change that causes different nodes to validate the same block differently is a critical defect.

---

10. SECURITY CONTRIBUTION POLICY

Security researchers should report vulnerabilities responsibly.

Do not publish exploitable details before the project has had a reasonable opportunity to investigate and coordinate remediation.

Security contributions should include:

- Affected component.
- Vulnerability description.
- Reproduction steps.
- Potential impact.
- Suggested mitigation.
- Relevant test cases.

Never submit real private keys, seed phrases, passwords, or user credentials.

A dedicated security reporting channel must be established before public mainnet launch.

---

11. CODING STANDARDS

All contributors must follow:

- Rust coding standards for Rust components.
- Kotlin coding standards for Android components.
- TypeScript coding standards for frontend components.
- Documented API conventions.
- Documented security requirements.

Code should be:

- Readable.
- Modular.
- Testable.
- Properly documented.

- Free from unnecessary dependencies.
- Consistent with existing architecture.

Unsafe Rust code requires a clear technical justification and additional review.

---

12. TESTING REQUIREMENTS

Contributions should include appropriate tests.

Depending on the change, testing may include:

- Unit tests.
- Integration tests.
- Property-based tests.
- Fuzz tests.
- Regression tests.
- Performance benchmarks.
- Wallet recovery tests.
- Network simulation tests.

Consensus and cryptographic changes require more extensive testing than ordinary interface changes.

All existing tests must pass before merging.

---

13. DEPENDENCY MANAGEMENT

New dependencies should be added carefully.

Contributors must:

1. Explain why a dependency is needed.
2. Check maintenance activity.
3. Review its license.
4. Evaluate security history.
5. Avoid unnecessary duplicate libraries.
6. Update dependency lockfiles.
7. Document important dependency changes.

Critical cryptographic dependencies require additional review.

---

14. DOCUMENTATION REQUIREMENTS

Documentation must be updated when a contribution changes:

- Architecture.
- API behavior.
- Consensus rules.
- Network parameters.
- Wallet behavior.
- Build procedures.
- Security requirements.
- User instructions.

Documentation should distinguish between:

- Proposed features.
- Implemented features.
- Tested features.
- Production-ready features.

Never describe an unimplemented feature as completed.

---

15. CODE REVIEW POLICY

Pull Requests should be reviewed before merging.

Reviewers should check:

- Correctness.
- Security.
- Code readability.
- Test coverage.
- Performance.
- Compatibility.
- Documentation.
- Dependency impact.

Consensus-related Pull Requests require specialized technical review.

Where practical, the author should not be the sole reviewer and approver of a critical change.

---

16. AUTOMATED CI CHECKS

The repository should eventually include automated checks for:

- Code formatting.
- Compilation.
- Unit tests.
- Integration tests.
- Dependency checks.
- Static analysis.
- Security scanning.
- Build verification.

Recommended Rust checks include:

- "cargo fmt --check"
- "cargo test"
- "cargo clippy"

The exact CI workflow must match the final repository structure and supported toolchain.

CI success does not replace manual security review.

---

17. PROHIBITED CONTRIBUTIONS

The following are not acceptable:

- Hidden private-key collection.
- Unauthorized wallet fund transfers.
- Backdoors.
- Malicious dependencies.
- Undocumented consensus changes.
- Hardcoded production secrets.
- Fake benchmark results.
- Misleading security claims.
- Deliberate introduction of vulnerabilities.
- Unauthorized changes to official release artifacts.

Such contributions must be rejected and investigated.

---

18. LICENSE AND INTELLECTUAL PROPERTY

Before accepting external contributions, SAECOIN must publish:

- An official project license.
- Contributor licensing terms, if required.
- Third-party dependency license requirements.

- Trademark and branding rules, if applicable.

Contributors must have the legal right to submit their code and documentation.

The final licensing model remains undecided.

---

19. COMMUNITY CONDUCT

All contributors should:

- Communicate respectfully.
- Provide constructive feedback.
- Avoid harassment.
- Respect different technical opinions.
- Report security problems responsibly.
- Avoid misleading users.
- Keep discussions focused on project development.

Technical disagreement should be resolved through evidence, testing, and documented decisions.

---

20. CONTRIBUTION ACCEPTANCE CRITERIA

A contribution may be accepted when:

1. It serves a clear project requirement.
2. It follows project coding standards.
3. It includes appropriate tests.
4. Existing tests pass.
5. Security implications are reviewed.
6. Documentation is updated where necessary.
7. Dependencies are justified.
8. Required approvals are obtained.

Acceptance does not automatically mean that a feature is ready for mainnet.

---

21. FINAL PRINCIPLE

SAECOIN should be developed through transparent, reviewed, and testable contributions.

No single contributor should be able to introduce an unreviewed critical consensus change into a production release.

The quality and security of SAECOIN depend on disciplined collaboration, independent review, and verifiable testing.

END OF DOCUMENT